Contents

Adaptive GPR Coverage — Technical Reference **1**
1. Notation and frames . . . 1
2. Scan region . . . 2
3. Boustrophedon decomposition . . . 2
4. Segment catalog and invalidation . . . 4
5. Lane pass merging . . . 5
6. Job ordering — ATSP sequencing . . . 6
7. Transit path planning — A* . . . 8
8. Path following and executed trace . . . 8
9. Swath coverage measurement . . . 9
10. Perception — log-odds mapping . . . 10
11. OARP-lite rank injection . . . 12
12. Behavior-tree orchestration . . . 12
13. Coverage metrics . . . 12
14. Visualization semantics . . . 13
15. Mission termination and failure compendium . . . 14
16. Default parameter summary . . . 15
17. Implementation index . . . 16

Adaptive GPR Coverage — Technical Reference

Methods-level specification. Defaults: `gpr_bringup/config/gpr_coverage.yaml`.
Equations use `$...$` / `$$...$$` (render on GitHub; in VS Code use a Markdown math extension).
See also: `ALGORITHM.md` · `SEQUENCING_AND_SEGMENTS.md` · PDFs: `./docs/verify-docs.sh`

---

1. Notation and frames

Symbol	Meaning
$\mathbf{p} = (x, y, \psi)$	Planar pose in frame `map`
$\mathcal{P} = \{\mathbf{p}_0, \cdots, \mathbf{p}_{n-1}\}$	Polyline (ordered waypoints)
$L(\mathcal{P})$	Arc length: $\sum_i \lVert\mathbf{p}_{i+1} - \mathbf{p}_i\rVert$
$s \in [0, L]$	Arc-length coordinate along a centerline
$\mathcal{R}$	User-defined scan region (rectangle or simple CCW polygon)
$\mathcal{R}_{\text{in}}$	Inset region used for lane generation
$\mathcal{G}$	Local occupancy grid on $\mathcal{R}$ bounds
c_{ij}	Cell cost at grid index (i, j); -1 = unknown

Frames: Gazebo `odom` is fixed at spawn via static `map → odom`. Planning, perception, and RViz use `map`. No SLAM: the scan boundary is configuration, not perception-derived.

Design constraints (assignment semantics):

1. No pre-built global map; only $\mathcal{R}$ plus rolling $\mathcal{G}$ from `/scan`.

2. $\mathcal{R}$ is fixed at mission start; perception invalidates **segments**, not the boundary polygon.
3. Initial plan is obstacle-free; blocking is applied online.
4. Adaptation is continuous: invalidate → split → reschedule → reconnect.

---

2. Scan region

2.1 Rectangle

$$\mathcal{R} = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$$

Default rectangle scenario: $[-4, 4] \times [-2.5, 2.5]$ m.

2.2 Polygon

Vertices $\mathbf{v}_0, \ldots, \mathbf{v}_{m-1}$ in counter-clockwise order. Validated by shoelace area $A > 0$:

$$A = \frac{1}{2} \sum_i (x_i y_{i+1} - x_{i+1} y_i)$$

2.3 Inset — $\mathcal{R}_{\mathbf{in}} = \mathbf{inset}(\mathcal{R}, d_{\mathbf{inset}})$

Default $d_{\text{inset}} = 0.60$ m (`coverage_inset`).

Axis-aligned rectangle: shrink each bound by d_{inset}.

General polygon: for each vertex $\mathbf{v}_i$, compute averaged inward normals from adjacent edges $\mathbf{e}_1 = \mathbf{v}_i - \mathbf{v}_{i-1}$, $\mathbf{e}_2 = \mathbf{v}_{i+1} - \mathbf{v}_i$, flip normal if $\mathbf{e}_1 \times \mathbf{e}_2 < 0$ (concave vertex), offset:

$$\mathbf{v}_i' = \mathbf{v}_i + d_{\text{inset}} \cdot \hat{\mathbf{n}}_i$$

Lanes are generated only inside $\mathcal{R}_{\text{in}}$.

---

3. Boustrophedon decomposition

Implementation: `BoustrophedonPlanner` (`gpr_planning`).

3.1 Scan direction

`scan_direction = "x"` (default): parallel lanes are lines of constant x; the robot drives along $\pm y$ within each chord.

`scan_direction = "y"`: lanes at constant y; drive along $\pm x$.

3.2 Lane coordinates

Along the spacing axis (e.g. x when `scan_direction = x`):

$$\mathcal{C} = \{c_k\} = \{x_{\min}, x_{\min} + \Delta_\ell, ...\} \cup \{x_{\max}\}$$

where $\Delta_\ell =$ `lane_spacing` (default 0.5 m). The maximum bound is always included.

3.3 Chords on a scan line

For each $c \in \mathcal{C}$, intersect polygon boundary with the scan line. Pair consecutive intersections $(a, b), (c, d), ...$ into chords. Keep chord $[a, b]$ if its midpoint lies inside $\mathcal{R}_{\text{in}}$ or on the boundary ($\leq 10^{-6}$ m).

Each chord receives a monotonic `lane_index` $= 0, 1, 2, ...$

3.4 Zig-zag orientation

For chord with endpoints $(x_0, y_0) \to (x_1, y_1)$:

$$\text{forward} = (\texttt{lane\_index} \bmod 2 = 0)$$

If `forward` is false, swap endpoints so drive direction alternates between adjacent lanes.

3.5 Waypoint discretization

For edge length $L_e = \|\mathbf{p}_1 - \mathbf{p}_0\|$:

$$N_{\text{steps}} = \max\left(1, \left\lceil \frac{L_e}{\Delta_{wp}} \right\rceil\right), \quad \mathbf{p}(t_i) = \mathbf{p}_0 + \frac{i}{N_{\text{steps}}}(\mathbf{p}_1 - \mathbf{p}_0), \quad \psi_i = \text{atan2}(\Delta y, \Delta x)$$

$\Delta_{wp} =$ `waypoint_spacing` (default 0.1 m).

3.6 Master segment catalog pieces

Each chord of length L is partitioned into

$$N_{\text{seg}} = \max\left(1, \left\lceil \frac{L}{\ell_{\text{seg}}} \right\rceil\right)$$

sub-intervals with $\ell_{\text{seg}} =$ `segment_length` (default 1.0 m). Sub-segment s spans $t \in [s/N_{\text{seg}}, (s + 1)/N_{\text{seg}}]$ along the chord, discretized as above.

Output segment fields:

Field	Value
`id`	`make_segment_id(lane_index, s)` — FNV-style hash
`lane_index`	chord index
`outcome`	`Pending`

Field	Value
`blocked`	`false`
`source`	`Baseline`

3.7 Full visualization path vs catalog

`generate()` additionally connects consecutive lane endpoints with straight connector edges for `/initial_coverage_path`. Connectors are **not** catalog segments; inter-lane motion uses A* transit.

---

4. Segment catalog and invalidation

Implementation: `SegmentCatalog`, `PathInvalidator`.

4.1 Segment state

`outcome`	Schedulable?	Counts as covered (metrics)?
`Pending`, `blocked=false`	Yes	No
`Pending`, `blocked=true`	Only if `schedule_blocked_probes=true` (default **false**)	No
`PartiallyCompleted`	**No** (tail not re-queued in current build)	Yes (covered arc only)
`Completed`	No	Yes
`Skipped`	No	No

Invariant: Segment lifecycle (outcome, `blocked`, `covered_intervals_m`, schedulability) is **identical** under `sequencer.use_boustrophedon: true` and `false`. Only job **visit order** differs.

Open segment: (`Pending` $\vee$ `PartiallyCompleted`) $\wedge \neg$`blocked` $\wedge\ |\mathcal{P}| \geq 2$.

For `PartiallyCompleted`, additionally require uncovered arc $\geq$ `min_split_length_m` (0.25 m) to remain open.

4.2 Collision model (`PathInvalidator`)

Footprint disk at world point (x, y): all cells within

$$r_{\text{cells}} = \left\lceil \frac{r_{\text{fp}}}{\Delta_{\text{grid}}} \right\rceil, \quad dx^2 + dy^2 \leq r_{\text{cells}}^2$$

where r_{fp} = `path_invalidator.footprint_radius` (0.22 m), Δ_{grid} = grid resolution.

Cell blocks if:

- out of map and `treat_unknown_as_free=false`, or

- $c < 0$ (unknown): blocks only when `block_only_observed_occupied=false` and unknown treated as occupied, or
- $c \geq \tau_{\text{obs}}$ (`obstacle_cost_threshold = 50`), unless within `boundary_ignore_margin` of polygon edge.

Default: `treat_unknown_as_free=true`, `block_only_observed_occupied=true` → only **observed** lethal cells block.

Edge sampling along segment $\mathbf{p}_0 \to \mathbf{p}_1$:

$$\Delta_s = \max(\text{segment\_sample\_spacing},\ 0.5 \cdot \Delta_{\text{grid}}), \quad N = \max\left(1, \left\lceil \frac{L}{\Delta_s} \right\rceil\right)$$

Sample $t = 0, 1/N, \ldots, 1$.

4.3 Free / blocked sub-polylines

Walk centerline vertices; flush contiguous collision-free runs. Yields `free_subpolylines` and `blocked_subpolylines`.

4.4 Split update (`update_blocked`)

Applied to each `Pending` segment when the grid changes:

Case	Action
`attempted && blocked`	Keep unchanged (frozen after failed attempt)
`OarpRank` source	Refresh `blocked` flag only
No free part	`blocked = true`
One free part, length $\geq L - 10^{-3}$	Replace centerline with trimmed free part
Multiple free parts or partial blockage	Spawn `Split` children with `baseline_id = effective_baseline_id(parent)`; drop fragments $<$ `min_split_length_m`

Split child id: `make_split_segment_id(baseline_id, ordinal)`.

---

5. Lane pass merging

Implementation: `BoustrophedonSequencer`.

Within each `lane_index`, sort segments by front-pose coordinate (y if `scan_direction=x`, else x) after orienting to lane drive direction.

Merge consecutive segments into one **pass** if any endpoint pair is within `merge_gap_m` (0.2 m):

$$\min_{\substack{e_a \in \{\text{front,back}\} \\ e_b \in \{\text{front,back}\}}} \|e_a - e_b\| \leq \text{merge\_gap\_m}$$

CoveragePass:

Field	Content
`centerline`	merged polyline along drive direction
`segment_ids`	constituent master/split ids
`job.segment_id`	`make_pass_job_id(lane_index, pass_ordinal)`
`job.covers`	`segment_ids`
`job.direction`	forward if `lane_index` even, else reverse

Passes are scheduling atoms alongside orphan segments not absorbed into any pass.

---

6. Job ordering — ATSP sequencing

Default mode: `sequencer.use_boustrophedon: true` → boustrophedon pass order. Set `false` for OR-Tools ATSP (heuristic fallback if OR-Tools unavailable).

6.1 Directed jobs

For each schedulable segment $\mathcal{P}$, unless direction is fixed:

$$\mathcal{J} = \{(\text{id}, \text{Forward}), (\text{id}, \text{Reverse})\}$$

Each directed job has entry pose $\mathbf{p}_{\text{entry}}$ and exit pose $\mathbf{p}_{\text{exit}}$ from `job_entry_pose` / `job_exit_pose` (respecting drive direction).

Pass pseudo-segments: one ATSP node per merged pass centerline (both directions if not fixed).

6.2 Transit cost for ordering (not execution path)

Important: ATSP uses a **cheap geometric estimate**, not A* search, because the solver evaluates $O(N^2)$ pairs:

$$d = \|\mathbf{p}_{\text{to}} - \mathbf{p}_{\text{from}}\|$$

$$\text{cost} = \begin{cases} d \cdot \lambda_{\text{block}} & \text{if straight segment } \mathbf{p}_{\text{from}} \to \mathbf{p}_{\text{to}} \text{ collides} \\ d & \text{otherwise} \end{cases}$$

λ_{block} = `astar.blocked_transit_penalty` (4.0). Collision check uses the same footprint disk model on the planning grid.

Matrix entry: `round(cost × 1000)`; non-finite → 10^9.

6.3 OR-Tools model

- Nodes: depot (robot pose) + directed jobs + optional home depot if return-home enabled.
- Arc costs: matrix above.

- **Disjunction:** for each physical `segment_id` with both Forward and Reverse nodes, `AddDisjunction({n_F, n_R}, penalty=10^8, max=1)` — at most one direction visited.
- Solver: `PATH_CHEAPEST_ARC`, time limit `sequencer.time_limit_sec` (2.0 s).
- On solver failure → `HeuristicAtspSolver` (greedy nearest-neighbor on segment ids).

6.4 Schedule queue

`recompute_schedule` produces ordered queue $\mathcal{Q} = [J_1, J_2, ...]$.

Next job to execute: `pop_next_job` returns $\mathcal{Q}_1$ (front), not a global re-optimization during motion.

When schedule rebuilds:

Trigger	Condition
`ScheduleNeedsRebuild`	Current job no longer actionable; queued job invalid; or queue empty while schedulable work exists; or OARP may inject
`force_replan`	After recovery skip
Interrupt replan	Active coverage job invalidated by map update (see §8)

Holding job during rebuild: overlapping jobs are removed from $\mathcal{Q}$; current job retained if still valid.

6.5 Reachability filter

If `sequencer.require_reachable_transit: true` (default), drop job J from $\mathcal{Q}$ when `is_job_reachable` is false:

$$d_{\text{entry}} = \|\mathbf{p}_{\text{robot}} - \mathbf{p}_{\text{entry}}\|$$

Condition	Reachable if
$d_{\text{entry}} < d_{\text{skip}}$	`job_is_followable` (coverage prefix exists on free grid)
else	A* path robot → entry exists **and** `is_transit_path_followable`

d_{skip} = `control.transit_skip_distance` (0.35 m).

If $\mathcal{Q}$ empty after filtering but schedulable segments remain: `block_unreachable_schedulable` marks segments `blocked` when **neither** forward nor reverse is reachable.

---

7. Transit path planning — A*

Used for: actual robot motion between jobs (once per job start), OARP batch validation, return-home.

Not used for: ATSP cost matrix (§6.2).

7.1 Algorithm

8-connected grid A* on `/local_occupancy_grid_planning`:

- State: cell (i, j)
- Step cost: Δ_{grid} (cardinal) or $\Delta_{\text{grid}}\sqrt{2}$ (diagonal)
- Heuristic: $h = \|\mathbf{p}_{\text{cell}} - \mathbf{p}_{\text{goal}}\|$
- Traversability: circular footprint r_{fp}, threshold τ_{obs}

Returns polyline from start to goal; fails (`nullopt`) if start/goal in collision, same cell only, open set exhausted, or post-check `path_is_traversable` fails.

7.2 When A* transit fails at job start

`ExecuteNextJob` returns **FAILURE** → recovery sequence:

`CancelControl → SkipCurrentJob (mark blocked) → RecomputeSchedule`

Job is not marked complete; `recovery_skip_count` increments (mission ends after 12 consecutive skips).

7.3 Transit vs coverage phases

$$\text{needs\_transit} = (d_{\text{entry}} \geq d_{\text{skip}}) \vee (\text{entry unknown})$$

Phase	Path	Trace tag
Transit	A* polyline to entry	`ExecutionTracePhase::Transit`
Coverage	`trim_polyline_ahead` + longest `free_subpolylines` prefix	`ExecutionTracePhase::Coverage`

After successful transit FollowPath, phase switches to coverage with a fresh follow goal.

During committed transit: `active_job_needs_replan` returns false — Nav2 completes current transit before replanning.

During coverage: if catalog invalidates the active job (`active_job_needs_replan`), cancel control, optionally release job, urgent `RecomputeSchedule`.

---

8. Path following and executed trace

Implementation: `Nav2FollowPathTracker` (`gpr_control`).

8.1 Path preparation

1. Find closest waypoint to robot.
2. If within `path_prune_distance` (0.15 m) of closest and not at end → advance start index by 1.
3. Prepend current robot pose; drop poses within 0.01 m of robot.
4. Require ≥ 2 poses.

8.2 Nav2 goal

Action `follow_path` with `controller_id=FollowPath`, `goal_checker_id=general_goal_checker`.

Success requires Nav2 result `SUCCEEDED` **and** (for coverage) swath threshold (§9).

8.3 Executed traces

Trace	Content	Decimation
`executed_coverage_trace_`	Coverage-phase poses only	0.05 m
`executed_transit_trace_`	Transit-phase poses	0.05 m
`mission_executed_trace_`	All phases	0.05 m

`flush_coverage_pose` force-appends final pose at coverage leg end (1e-4 m minimum spacing).

Published on `/coverage_executed_path`.

---

9. Swath coverage measurement

Implementation: `compute_swath_coverage` (`gpr_common`).

9.1 Point projection

For driven pose $\mathbf{p}$ and centerline $\mathcal{C}$, find closest projection on each segment, keep minimum-distance segment with arc coordinate s, lateral offset $d_\perp$, heading error $\Delta\psi$.

9.2 On-swath acceptance

Pose accepted iff:

$$d_\perp \leq d_{\max} \quad (0.22 \text{ m})$$

$$\min(|\Delta\psi|,\ |\pi - |\Delta\psi||) \leq \psi_{\max} \quad (35°)$$

Bidirectional allowance permits reverse-lane driving on the same geometric centerline.

9.3 Covered arc intervals

For each accepted sample at s, stamp:

$$[s-h,\ s+h], \quad h = \text{sample\_half\_width\_m} = 0.025 \text{ m}$$

Bridge consecutive accepted samples: $[\min(s_{k-1}, s_k), \max(s_{k-1}, s_k)]$.

Merge overlapping intervals; define:

$$f_{\text{cov}} = \frac{\text{union\_length(intervals)}}{L(\mathcal{C})}$$

9.4 Pass-level projection

For merged pass centerline $\mathcal{C}_{\text{pass}}$ and constituent segment $\mathcal{C}_s$, map pass intervals onto segment arc by endpoint projection (`coverage_for_segment_from_pass`).

Pass-level updates: when a pass job completes, only segments matching `job.covers` (and their split children via `baseline_id`) receive coverage. Pass shortcut at 85% marks those cover ids `Completed`.

9.5 Completion thresholds

Threshold	Default	Outcome
`min_complete_fraction`	0.85	`Completed`
`min_partial_fraction`	0.35	`PartiallyCompleted` if `partial_enabled`
else	—	stays `Pending` (intervals still stored)

Job satisfaction (`job_completion_satisfied`):

- **Pass job:** $f_{\text{cov}} \geq 0.85$ on pass centerline, OR partial enabled and $f_{\text{cov}} \geq 0.35$.
- **Non-pass multi-cover: all** matching segments must satisfy the same (AND).

Pass shortcut: if pass $f_{\text{cov}} \geq 0.85$, all in-scope lane segments → `Completed` with interval $[0, L(\mathcal{C}_s)]$.

`PartiallyCompleted` segments are **not** re-scheduled in the current configuration.

---

10. Perception — log-odds mapping

Implementation: `OccupancyGridMapper`, `PerceptionBridge`.

10.1 Grid layout

$$W = \left\lceil \frac{x_{\max} - x_{\min}}{\Delta_{\text{grid}}} \right\rceil, \quad H = \left\lceil \frac{y_{\max} - y_{\min}}{\Delta_{\text{grid}}} \right\rceil$$

Origin $(x_{\min}, y_{\min})$. Default $\Delta_{\text{grid}} = 0.05$ m.

10.2 Log-odds update

Prior per cell: $L_0 = 0$ (50% occupancy). Hit/miss increments:

$$L_{\text{hit}} = \ln \frac{p_{\text{hit}}}{1 - p_{\text{hit}}}, \quad L_{\text{miss}} = \ln \frac{p_{\text{miss}}}{1 - p_{\text{miss}}}$$

Defaults $p_{\text{hit}} = 0.85$, $p_{\text{miss}} = 0.35$:

$$L_{\text{hit}} \approx 1.736, \quad L_{\text{miss}} \approx -0.619$$

Per observation:

$$L \leftarrow \text{clamp}(L + \Delta L,\ -L_{\max},\ L_{\max}), \quad L_{\max} = 5.0$$

10.3 Ray integration (Bresenham)

For each valid `/scan` beam transformed to `map`:

1. All cells along ray **except** endpoint: $\Delta L = L_{\text{miss}}$.
2. Endpoint if range hit: $\Delta L = L_{\text{hit}}$.
3. **Hit disk:** all cells with $dx^2 + dy^2 \leq r_{\text{hit}}^2$ receive L_{hit}, where $r_{\text{hit}} = \lceil 0.15/\Delta_{\text{grid}} \rceil$.

10.4 Export to cost grid

For observed cells:

$$p = \frac{e^L}{1 + e^L}, \quad p_{\%} = \text{round}(100p)$$

Condition	Cost
Unobserved	-1
$p_{\%} \geq \tau_{\text{export}}$ (50)	50 (lethal)
else	$\text{clamp}(p_{\%}, 0, 100)$

10.5 Inflation (visualization grid only)

Hard-occupied cells ($c \geq 50$) inflate to cost 100 in disk $r_{\text{inf}} = \lceil 0.35/\Delta_{\text{grid}} \rceil$.

Topic	Grid	Inflation
`/local_occupancy_grid`	threshold + inflate	Yes
`/local_occupancy_grid_planning`	threshold only	**No**

Mission invalidation, A*, and Nav2 use the **planning** grid (single inflation layer in Nav2 params).

---

11. OARP-lite rank injection

When `oarp_lite.enabled` and **no** schedulable baseline/split segments remain on a lane, but open work may exist elsewhere:

1. For each lane centerline with no open catalog work, extract `free_subpolylines`.
2. Skip chunks $<$ `min_rank_length_m` (0.35 m) or overlapping any `Completed` segment on lane (0.2 m proximity).
3. Split chunk into $\lceil \text{length}/\ell_{\text{seg}} \rceil$ linear pieces; discretize at `waypoint_spacing`.
4. Inject as `OarpRank` segments; cap `max_replan_generations` (2) per mission.

Before accepting a batch: A* from robot to first rank entry must succeed (else entire batch rejected).

---

12. Behavior-tree orchestration

Tree: `coverage_mission.xml`. Tick rate: 2 Hz.

```
WaitForMap
GenerateInitialCoverage
loop until HasPendingJobs fails:
  UpdateSegmentCatalog
  if ScheduleNeedsRebuild: RecomputeSchedule
  ExecuteNextJob OR [CancelControl → SkipCurrentJob → RecomputeSchedule]
MissionComplete
ReturnToStart (optional)
```

Node	Semantics
`WaitForMap`	Blocks until perception has a grid
`GenerateInitialCoverage`	`planner.generate()` + `catalog.initialize(segments)`; freeze baseline reporter
`UpdateSegmentCatalog`	`catalog.update_blocked(latest_grid)`
`HasPendingJobs`	`has_work_remaining()` and `recovery_skip_count < 12`
`ExecuteNextJob`	`pop_next_job` → transit/coverage FollowPath → record coverage → complete or fail

Mission ends when no work remains, reachability exhausted, or 12 recovery skips.

---

13. Coverage metrics

Baseline (frozen at start):

$$L_{\text{plan}} = \sum_{s \in \text{baseline}} L(\mathcal{C}_s), \quad A_{\text{plan}} = L_{\text{plan}} \cdot w_{\text{swath}}$$

w_{swath} = `lane_spacing` (0.5 m).

Rollup per baseline master (handles splits):

Outcome	Length credit
`Completed`	full L
`PartiallyCompleted`	covered arc → completed; uncovered → pending
`Pending`, blocked	blocked
`Pending`, free	pending
`Skipped`	skipped

Implicit blocked gap: $\max(0, L_{\text{baseline}} - \sum L_{\text{children}})$.

Percent KPIs: pct $= 100 \cdot \text{part}/L_{\text{plan}}$ for completed, partial, blocked, skipped, remaining, plan-retention.

---

14. Visualization semantics

14.1 `/updated_coverage_path_markers` (primary)

Drawn on **frozen baseline geometry** from the reporter, subdivided by live grid collision (`free_subpolylines` / `blocked_subpolylines`). Colors from segment outcome + collision overlay:

Color	RGB (approx)	Meaning
Green	(0.1, 0.8, 0.1)	`Completed`
Gold / yellow	(0.95, 0.75, 0.1)	`PartiallyCompleted`
Orange	(1.0, 0.5, 0.0)	`Pending`, collision-free
Amber	(0.9, 0.4, 0.1)	`Pending`, `blocked`, not yet attempted
Red	(0.9, 0.1, 0.1)	Blocked on grid overlay, or attempted+blocked
Purple	(0.6, 0.1, 0.8)	`Skipped`
Blue	(0.2, 0.6, 1.0)	`OarpRank` source

Grid-blocked portions are drawn **red** even when catalog outcome is `Completed` or `PartiallyCompleted`.

Catalog vs RViz invariants:

- **Catalog `outcome` is authoritative** for free-fragment colors (orange / gold / green). RViz does **not** upgrade `Pending` to gold from coverage intervals alone.
- **`covered_intervals_m` with `Pending` outcome cannot occur** after a coverage update; any non-zero drive progress becomes `PartiallyCompleted` and is excluded from `schedulable_segments()`.
- **Grid collision overlay is authoritative for red/amber** on subpolylines (`blocked_subpolylines`), regardless of catalog outcome.
- Interval-based shading (0.85 / 0.35) applies **per baseline master arc** (each 1 m frozen piece), not whole-lane rollup. Coverage intervals are mapped only from catalog pieces sharing the same `baseline_id`.

14.2 Other topics

Topic	Content
`/initial_coverage_path`	Cyan frozen boustrophedon (connectors included)
`/coverage_transit_path`	Blue active A* transit
`/coverage_executed_path`	Driven trajectory (all phases)
`/updated_coverage_path_strips`	Orange remaining schedulable centerlines
`/coverage_swath_heatmap`	Green cubes = covered arc; orange = uncovered (off by default)
`/local_occupancy_grid`	Inflated occupancy (off by default)
`/local_occupancy_grid_planning`	Planning threshold grid (off by default)

Do not enable `/updated_coverage_path` in RViz — poses are chained in schedule order, producing spurious diagonals between unrelated swaths.

---

15. Mission termination and failure compendium

Question	Answer
How is the **next segment** chosen?	Front of ATSP-ordered queue $\mathcal{Q}$ after reachability filter; not re-solved every step.
What **metric** orders jobs?	Penalized Euclidean transit cost between job entry/exit poses on the planning grid; OR-Tools minimizes tour length.
Is A* used for ordering?	**No** — only for executed transit and reachability checks.
Is the path replanned online until goal?	**Schedule** replans when catalog/validity changes; **transit** path is one A* per job; **coverage** replans prefix when map invalidates during coverage (not during committed transit).
A* fails to entry?	Job fails → marked blocked → schedule recomputed.
Nav2 fails mid-leg?	Same recovery; increment skip counter.
Coverage leg succeeds but $f_{\text{cov}} < 0.35$?	FAILURE → blocked → replan.
$0.35 \leq f_{\text{cov}} < 0.85$?	Partial outcome stored; job can succeed if partial threshold met; segment not re-queued.
Segment fully blocked by grid?	`blocked=true`; not scheduled unless probes enabled.
Partial split fragments?	Children ≥ 0.25 m scheduled independently.
All jobs unreachable?	Segments marked blocked; `reachability_exhausted` → mission winds down.

Question	Answer
Lanes exhausted, gaps remain?	OARP injects up to 2 generations of rank segments.
Does perception shrink the scan polygon?	**No.** Only segment centerlines are invalidated.
Global map used?	**No.** $\mathcal{R}$ + rolling $\mathcal{G}$ only.

---

16. Default parameter summary

All tunables live in `gpr_bringup/config/gpr_coverage.yaml` under `gpr_mission.ros__parameters`. Launch-only keys (`launch.*`) are read by `gpr_coverage.launch.py` from the same file.

Parameter	Value	Role
`lane_spacing`	0.5 m	Parallel lane spacing
`waypoint_spacing`	0.1 m	Centerline discretization
`coverage_inset`	0.60 m	Boundary clearance
`segment_length`	1.0 m	Catalog piece length
`scan_direction`	`"x"`	Lane orientation
`sequencer.use_boustrophedon`	`true`	Lane-order vs ATSP
`sequencer.merge_gap_m`	0.2 m	Pass merging
`sequencer.require_reachable_transit`	`true`	Reachability filter
`sequencer.time_limit_sec`	2.0 s	OR-Tools budget
`sequencer.atsp.direction_disjunction_penalty`	1e8	One direction per segment
`sequencer.atsp.unreachable_transit_cost`	1e9	Infeasible ATSP edge cost
`astar.blocked_transit_penalty`	4.0	ATSP blocked-edge penalty
`path_invalidator.footprint_radius`	0.22 m	Invalidation clearance
`path_invalidator.map_inflation_radius`	0.0 m	Extra invalidation inflation
`path_invalidator.obstacle_cost_threshold`	50	Lethal cost
`coverage.min_complete_fraction`	0.85	Completion
`coverage.min_partial_fraction`	0.35	Partial
`coverage.lateral_max_m`	0.22 m	Swath lateral gate
`coverage.heading_max_deg`	35°	Swath heading gate
`planning.executed_trace_step_m`	0.05 m	Mission/coverage trace decimation
`planning.flush_pose_min_step_m`	1e-4 m	Force-append at leg end
`mission.max_recovery_skips`	12	Consecutive Nav2 failures before abort
`control.transit_skip_distance`	0.35 m	Direct coverage if closer

Parameter	Value	Role
control.path_prune_distance	0.15 m	FollowPath prune
local_mapping.resolution	0.05 m	Grid cell size
local_mapping.prob_hit / prob_miss	0.85 / 0.35	Log-odds increments
local_mapping.occupied_export_threshold	50	Lethal export
local_mapping.inflation_radius	0.35 m	Viz inflation
oarp_lite.max_replan_generations	2	Rank injection cap
oarp_lite.overlap_dist_m	0.2 m	Completed-rank overlap gate
segment_catalog.mark_completed_overlap_dist_m	0.25 m	Legacy overlap completion
segment_catalog.mark_completed_min_overlap_fraction	0.55	Legacy overlap completion
visualization.complete_fraction	0.85	RViz baseline arc green
visualization.partial_fraction	0.35	RViz baseline arc gold
visualization.marker_line_width	0.05 m	Segment marker thickness
visualization.*_topic	see yaml	All publisher topic names

---

17. Implementation index

Component	Primary source files
Boustrophedon	gpr_planning/src/boustrophedon_planner.cpp
Polygon geometry	gpr_common/src/polygon_geometry.cpp, scan_region.cpp
Pass merging	gpr_planning/src/boustrophedon_sequencer.cpp
Invalidation	gpr_planning/src/path_invalidator.cpp
Catalog	gpr_planning/src/segment_catalog.cpp
A*	gpr_planning/src/astar_grid_planner.cpp
ATSP	gpr_planning/src/or_tools_atsp_solver.cpp, heuristic_atsp_solver.cpp
Planning engine	gpr_planning/src/planning_engine.cpp
Swath coverage	gpr_common/src/swath_coverage.cpp
Perception	gpr_perception/src/occupancy_grid_mapper.cpp
Mission BT	gpr_mission/bt_xml/coverage_mission.xml, mission_behaviors.cpp
Control	gpr_control/src/nav2_follow_path_tracker.cpp
Metrics	gpr_metrics/src/coverage_accountant.cpp
RViz IO	gpr_planning/src/ros_io_facade.cpp